

ProofPay

Version professionnelle complète – cahier des charges, architecture, diagrammes et SQL

Objet : fournir un document unique, structuré et directement exploitable pour cadrer, concevoir et lancer un MVP ProofPay, une plateforme d'escrow digital destinée aux transactions entre particuliers et petits vendeurs/prestataires.

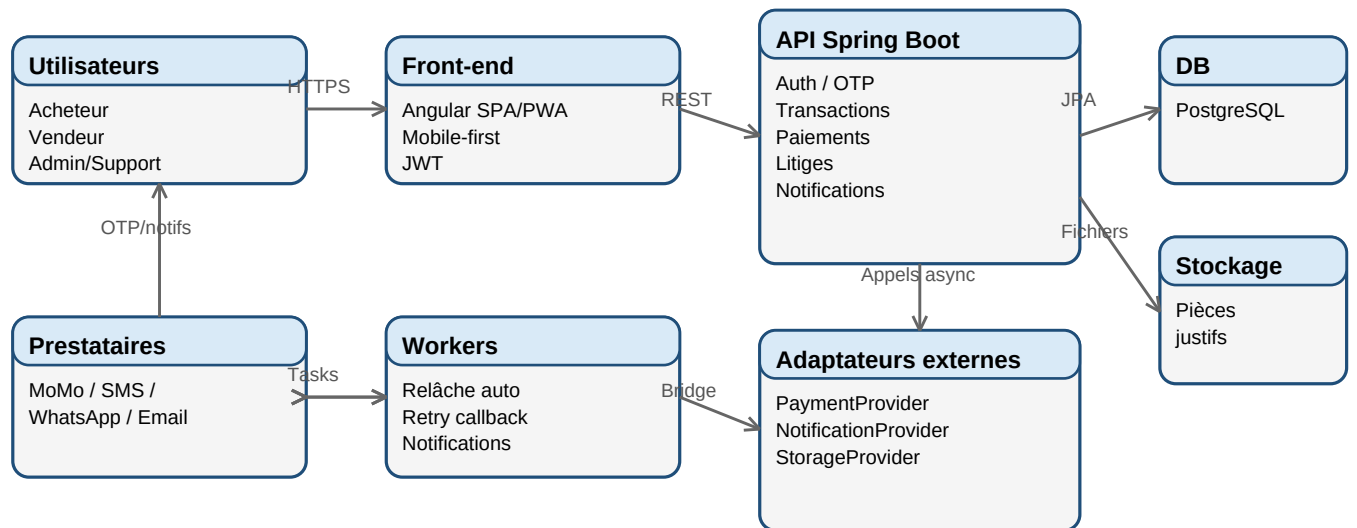
1. Résumé exécutif

- ProofPay répond à un problème terrain : les arnaques et la défiance lors des ventes ou prestations réalisées à distance.
- Le service repose sur une logique d'escrow : l'acheteur paie, le montant reste bloqué, puis il est libéré lorsque la livraison ou l'exécution est validée.
- Le MVP doit viser un parcours très simple, mobile-first, centré sur quatre étapes : créer, payer, livrer, confirmer.
- Le projet doit démarrer avec un adaptateur de paiement simulé ou semi-manuel, afin de valider le marché avant d'industrialiser les intégrations réelles.
- La valeur produit repose sur la simplicité, la traçabilité et une gestion claire des litiges.

2. Spécifications fonctionnelles

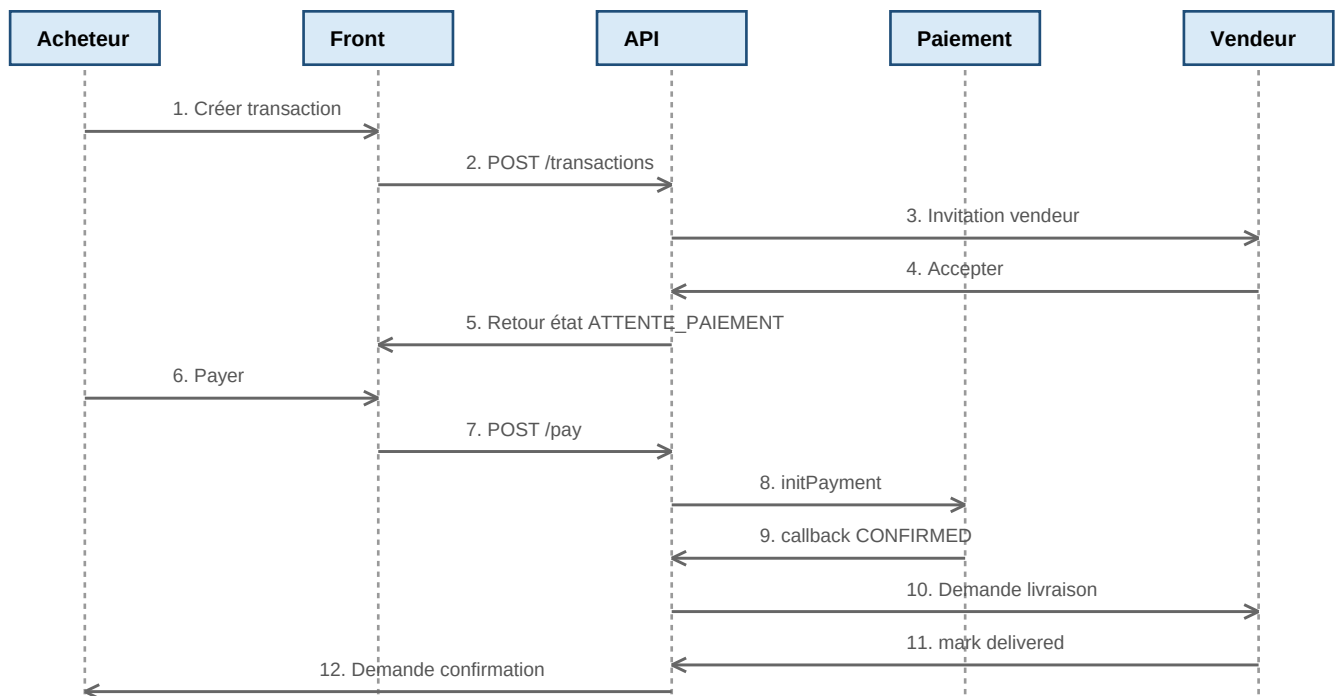
Domaine	Points clés
Périmètre MVP	Créer transaction, acceptation vendeur, paiement, livraison, confirmation, litige, notifications
Acteurs	Acheteur, vendeur, admin/support, prestataires externes
Règles métier	Paiement seulement après acceptation, litige gèle workflow, relâche auto pilotée par délai
Indicateurs	Taux de conversion, taux de clôture sans litige, délai moyen de résolution

3. Diagramme d'architecture

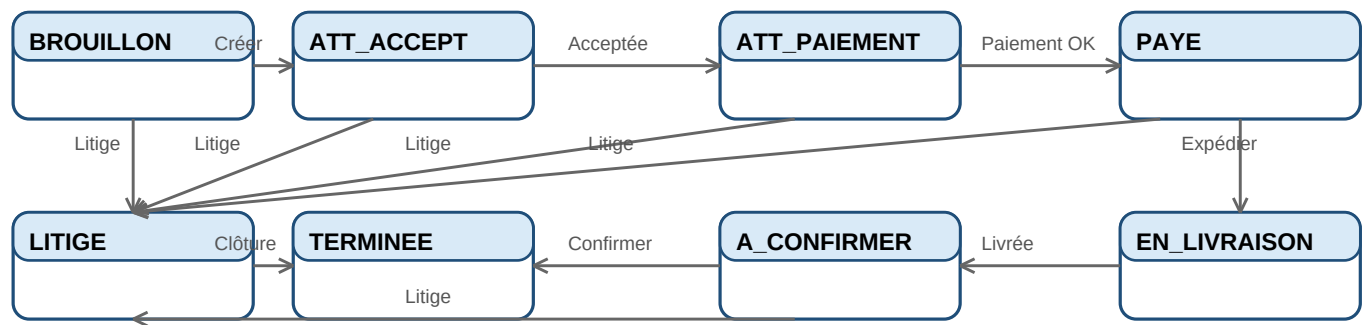


Ce schéma illustre la cible technique recommandée pour un MVP robuste mais simple : Angular + Spring Boot + PostgreSQL + adaptateurs externes.

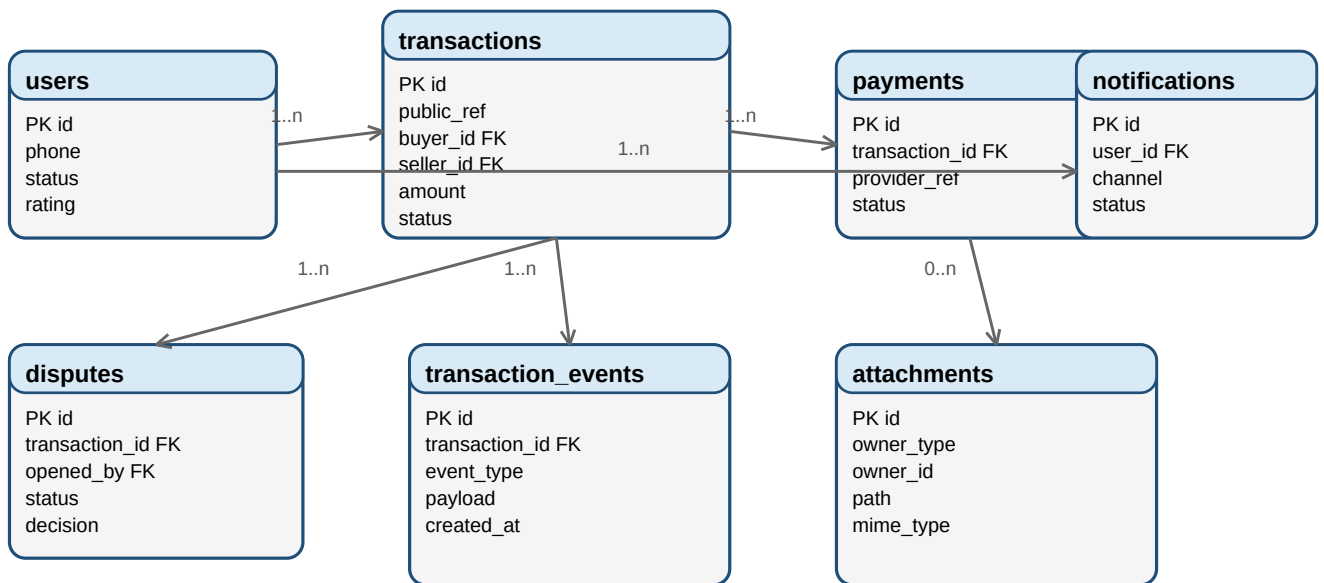
4. Séquence nominale



5. Machine d'états



6. MCD / ERD simplifié



7. API REST recommandée

Méthode	URI	Description
POST	/api/auth/request-otp	Demande OTP
POST	/api/auth/verify-otp	Validation OTP / JWT
GET	/api/users/me	Profil courant
POST	/api/transactions	Créer transaction
POST	/api/transactions/{id}/accept	Accepter
POST	/api/transactions/{id}/reject	Refuser
POST	/api/transactions/{id}/pay	Init paiement
POST	/api/payments/callback/{provider}	Callback prestataire
POST	/api/transactions/{id}/mark-delivered	Déclarer livraison
POST	/api/transactions/{id}/confirm	Confirmation finale
POST	/api/transactions/{id}/open-dispute	Ouvrir litige
POST	/api/disputes/{id}/decision	Décision support

8. Modèle de données relationnel

Table	Rôle
users	Comptes et réputation
user_verifications	OTP / KYC / vérifications
transactions	Cœur métier de la transaction
transaction_events	Journal d'audit métier

Table	Rôle
payments	Paiements et rapprochements
payment_attempts	Historique détaillé des essais
disputes	Litiges et décisions
attachments	Pièces justificatives
notifications	Historique des messages envoyés
admin_settings	Paramètres métier

9. Script SQL PostgreSQL

```
-- =====
-- PROOFPAY - PostgreSQL DDL v1
-- =====
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TYPE user_status AS ENUM ('PENDING_VERIFICATION', 'ACTIVE', 'SUSPENDED', 'BLOCKED');
CREATE TYPE transaction_status AS ENUM (
    'BROUILLON', 'EN_ATTENTE_ACCEPTATION', 'REFUSEE', 'EN_ATTENTE_PAIEMENT', 'PAYE',
    'EN_LIVRAISON', 'A_CONFIRMER', 'RELACHE_AUTO', 'LITIGE', 'TERMINEE', 'REMBOURSEE', 'ANNULEE', 'EXPIREE'
);
CREATE TYPE payment_status AS ENUM ('INITIATED', 'PENDING', 'CONFIRMED', 'FAILED', 'REVERSED');
CREATE TYPE dispute_status AS ENUM ('OPEN', 'UNDER_REVIEW', 'RESOLVED', 'REJECTED');
CREATE TYPE confirmation_mode AS ENUM ('BUTTON', 'OTP', 'CODE_SECRET');
CREATE TYPE notification_channel AS ENUM ('SMS', 'EMAIL', 'WHATSAPP', 'PUSH');

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phone VARCHAR(20) NOT NULL UNIQUE,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    display_name VARCHAR(150),
    email VARCHAR(150),
    photo_url TEXT,
    preferred_language VARCHAR(10) DEFAULT 'fr',
    status user_status NOT NULL DEFAULT 'PENDING_VERIFICATION',
    rating NUMERIC(3,2) DEFAULT 0,
    transactions_count INTEGER NOT NULL DEFAULT 0,
    disputes_opened_count INTEGER NOT NULL DEFAULT 0,
    disputes_lost_count INTEGER NOT NULL DEFAULT 0,
    last_login_at TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_verifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    verification_type VARCHAR(30) NOT NULL,
    status VARCHAR(30) NOT NULL,
    verification_value VARCHAR(255),
    expires_at TIMESTAMP,
    verified_at TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    public_ref VARCHAR(30) NOT NULL UNIQUE,
```

```
buyer_id UUID NOT NULL REFERENCES users(id),
seller_id UUID NOT NULL REFERENCES users(id),
title VARCHAR(200) NOT NULL,
description TEXT,
category_code VARCHAR(50),
currency CHAR(3) NOT NULL DEFAULT 'XOF',
amount NUMERIC(18,2) NOT NULL CHECK (amount > 0),
fees NUMERIC(18,2) NOT NULL DEFAULT 0,
total_amount NUMERIC(18,2) GENERATED ALWAYS AS (amount + fees) STORED,
status transaction_status NOT NULL DEFAULT 'BROUILLON',
confirmation_mode confirmation_mode NOT NULL DEFAULT 'BUTTON',
confirmation_secret_hash VARCHAR(255),
delivery_deadline TIMESTAMP,
auto_release_at TIMESTAMP,
paid_at TIMESTAMP,
delivered_at TIMESTAMP,
confirmed_at TIMESTAMP,
completed_at TIMESTAMP,
cancelled_at TIMESTAMP,
metadata JSONB,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT ck_buyer_seller_different CHECK (buyer_id <> seller_id)
);

CREATE TABLE transaction_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
  event_type VARCHAR(50) NOT NULL,
  previous_status transaction_status,
  new_status transaction_status,
  actor_user_id UUID REFERENCES users(id),
  payload JSONB,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
  provider_code VARCHAR(50) NOT NULL,
  channel VARCHAR(30),
  request_ref VARCHAR(100) NOT NULL UNIQUE,
  provider_ref VARCHAR(100),
  amount NUMERIC(18,2) NOT NULL CHECK (amount > 0),
  currency CHAR(3) NOT NULL DEFAULT 'XOF',
  status payment_status NOT NULL DEFAULT 'INITIATED',
  failure_reason TEXT,
  callback_payload JSONB,
```

```
    paid_at TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE payment_attempts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id UUID NOT NULL REFERENCES payments(id) ON DELETE CASCADE,
    attempt_no INTEGER NOT NULL,
    result_code VARCHAR(50),
    result_message TEXT,
    raw_payload JSONB,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(payment_id, attempt_no)
);

CREATE TABLE disputes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    transaction_id UUID NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
    opened_by UUID NOT NULL REFERENCES users(id),
    reason_code VARCHAR(50) NOT NULL,
    reason_details TEXT,
    status dispute_status NOT NULL DEFAULT 'OPEN',
    decision_code VARCHAR(50),
    decision_comment TEXT,
    resolved_by UUID REFERENCES users(id),
    opened_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    resolved_at TIMESTAMP
);

CREATE TABLE attachments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_type VARCHAR(30) NOT NULL,
    owner_id UUID NOT NULL,
    original_name VARCHAR(255) NOT NULL,
    stored_name VARCHAR(255) NOT NULL,
    mime_type VARCHAR(100) NOT NULL,
    size_bytes BIGINT NOT NULL,
    storage_path TEXT NOT NULL,
    uploaded_by UUID REFERENCES users(id),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    transaction_id UUID REFERENCES transactions(id) ON DELETE CASCADE,
    channel notification_channel NOT NULL,
```

```
template_code VARCHAR(50) NOT NULL,
destination VARCHAR(255) NOT NULL,
payload JSONB,
status VARCHAR(30) NOT NULL DEFAULT 'PENDING',
provider_message_id VARCHAR(100),
sent_at TIMESTAMP,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE admin_settings (
    setting_key VARCHAR(100) PRIMARY KEY,
    setting_value TEXT NOT NULL,
    scope VARCHAR(50) DEFAULT 'GLOBAL',
    description TEXT,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_transactions_buyer ON transactions(buyer_id);
CREATE INDEX idx_transactions_seller ON transactions(seller_id);
CREATE INDEX idx_transactions_status ON transactions(status);
CREATE INDEX idx_transactions_created_at ON transactions(created_at);
CREATE INDEX idx_transactions_auto_release_at ON transactions(auto_release_at);
CREATE INDEX idx_transaction_events_tx ON transaction_events(transaction_id, created_at);
CREATE INDEX idx_payments_tx ON payments(transaction_id);
CREATE INDEX idx_payments_provider_ref ON payments(provider_ref);
CREATE INDEX idx_disputes_tx ON disputes(transaction_id);
CREATE INDEX idx_notifications_user ON notifications(user_id, created_at);

INSERT INTO admin_settings(setting_key, setting_value, description) VALUES
('SERVICE_FEE_PERCENT', '2.00', 'Commission standard en pourcentage'),
('SERVICE_FEE_MIN_XOF', '100', 'Commission minimale en XOF'),
('DEFAULT_CONFIRMATION_DELAY_HOURS', '48', 'Délai avant relâche automatique'),
('MAX_ATTACHMENT_MB', '10', 'Taille max d''une pièce jointe');
```


10. Plan projet MVP

Lot	Objectif	Livrables
Lot 0	Cadrage	Vision, règles métier, backlog
Lot 1	Socle technique	Front/back, auth OTP, base de données
Lot 2	Noyau transaction	Création, acceptation, statuts, audit
Lot 3	Paiement	Mock provider, callback, rapprochement
Lot 4	Livraison / confirmation	mark-delivered, confirm, auto-release
Lot 5	Litiges / support	Workflow, pièces, écran admin
Lot 6	Industrialisation	Logs, sécurité, monitoring, recette pilote

- Commencer avec un provider de paiement simulé pour valider rapidement le besoin.
- Ne pas surcharger le MVP : le cœur est la confiance transactionnelle, pas la marketplace.
- Préparer le chantier réglementaire avant toute mise à l'échelle impliquant de vrais flux financiers bloqués.
- Conserver un support humain au démarrage pour les litiges et les cas limites.

Fin du dossier. Ce PDF constitue une base professionnelle exploitable pour la conception, la priorisation et le lancement d'un MVP ProofPay.